Smart Camera & Video Surveillance System

A Deep Learning based Criminal Recognition and Tracking System

60% Progress Report

Date: 07 September 2026

Phase: Setup + Detection + Recognition + Alerts + Logging

Status: 60% Complete

Final Year B.Tech (CSE) Major Project

Table of Contents

1. Project Overview (60% Complete)
2. System Architecture
3. Technologies & Libraries Used
4. Project Folder Structure
5. Module 1: Face Detection (face_detector.py)
6. Module 2: Face Recognition (face_recognizer.py)
7. Module 3: Database Management (database.py)
8. Module 4: Email Alert System (email_alert.py)
9. Module 5: Detection Logger (detection_logger.py)
10. Module 6: Main Surveillance (surveillance.py)
11. Module 7: Face Registration (register_faces.py)
12. How to Run the Complete System
13. How Face Recognition Works (CNN + Embeddings)
14. Testing Results
15. Progress Summary & Remaining Work

1. Project Overview (60% Complete)

This project builds a Smart Surveillance System that makes CCTV cameras intelligent. Instead of just recording footage, the camera actively watches the video feed, detects faces, recognizes known individuals, and sends instant email alerts when a match is found.

At 60% completion, the system has the following working components:

- Real-time face detection from webcam using OpenCV Haar Cascade
- Face recognition using dlib + face_recognition library (128-dim embeddings)
- SQLite database for storing face encodings and detection logs
- Automated email alert system with snapshot attachment
- CSV-based detection logging with statistics
- Interactive face registration system (add new faces to database)
- Main integrated surveillance system with all features wired together

Progress Breakdown

Component	Status	Completion
Face Detection (Haar Cascade)	DONE	100%
Face Recognition (CNN Embeddings)	DONE	100%
Database (SQLite)	DONE	100%
Email Alert System	DONE	100%
Detection Logging	DONE	100%
Face Registration UI	DONE	100%
Main Integration	DONE	100%
Flask Web Dashboard	PENDING	0%
Multi-camera Support	PENDING	0%
Behavior Analysis	PENDING	0%
Testing & Documentation	PENDING	0%

2. System Architecture

The system follows a modular pipeline architecture. Each component is a separate Python module that can be used independently or combined into the main surveillance system.

```
CAMERA FEED
    |
    v
FRAME CAPTURE (OpenCV)
    |
    v
FACE DETECTION (Haar Cascade)
    |
    v
FACE ENCODING (dlib/face_recognition)
    |--- 128-dimensional vector
    |
    v
COMPARE WITH DATABASE (SQLite)
    |--- Distance < 0.6 = MATCH
    |--- Distance >= 0.6 = UNKNOWN
    |
    +--> MATCH FOUND
    |      |-- Save snapshot
    |      |-- Log detection
    |      |-- Send email alert
    |
    +--> NO MATCH
           |-- Log as unknown
           |-- Continue monitoring
```

3. Technologies & Libraries Used

Technology	Version	Role in Project
Python	3.8.10	Main programming language
OpenCV	4.8.1	Video capture, face detection, image processing
dlib	20.0.1	Face encoding, facial landmark detection
face_recognition	Latest	High-level face recognition API
NumPy	1.24.4	Array operations, distance calculations
SQLite	Built-in	Face database and detection logs
smtplib	Built-in	Email alert via SMTP protocol
Haar Cascade	XML	Pre-trained face/eye detection model

How face_recognition Works Under the Hood

The face_recognition library uses dlib's deep learning model. It works in two steps:

Step 1 - Face Detection: Uses HOG (Histogram of Oriented Gradients) or CNN to locate faces in the image. Returns the bounding box coordinates.

Step 2 - Face Encoding: Takes the detected face and passes it through a deep neural network (ResNet trained on ~3 million faces). Outputs a 128-dimensional vector (embedding) that uniquely represents that face. Similar faces have similar vectors.

4. Project Folder Structure

```
major_project/
|
|-- config/
|   |-- settings.py              # All configuration constants
|
|-- src/
|   |-- face_detector.py         # Haar Cascade face detection
|   |-- face_recognizer.py       # CNN-based face recognition
|   |-- database.py              # SQLite database operations
|   |-- email_alert.py           # SMTP email alerts
|   |-- detection_logger.py      # CSV logging system
|   |-- surveillance.py          # Main integrated system
|   |-- register_faces.py        # Add faces to database
|   |-- webcam_capture.py        # Week 1 webcam detection
|   |-- image_face_detection.py  # Week 1 image detection
|   |-- dataset_collector.py     # Week 1 dataset collection
|   |-- test_recognition.py      # Recognition test utility
|
|-- models/
|   |-- haarcascade_frontalface_default.xml
|   |-- haarcascade_eye.xml
|
|-- known_faces/                 # Known face images
|-- snapshots/                   # Detection snapshots
|-- dataset/                     # Collected face datasets
|-- logs/
|   |-- faces.db                 # SQLite database
|   |-- detection_log.csv        # CSV detection log
|-- pdf/                         # Documentation
```

5. Module 1: face_detector.py (Week 1)

Purpose

Handles basic face detection using OpenCV's Haar Cascade classifier. Detects faces and eyes in images, webcam feed, or video files.

Class: FaceDetector

- __init__(): Loads Haar Cascade XML files for face and eye detection
- detect_faces(frame): Detects all faces in a frame, returns list of results
- draw_detections(frame, detections): Draws green bounding boxes on frame
- save_snapshot(frame, detection): Saves cropped face image to snapshots/

How Haar Cascade Detection Works

1. Convert frame to grayscale
2. Scale image at multiple sizes (scaleFactor=1.3)
3. Slide a window across the image
4. Check each window using pre-trained Haar features
5. If enough features match, classify as face
6. Merge overlapping detections (minNeighbors=5)

6. Module 2: face_recognizer.py

Purpose

The core recognition module. Converts detected faces into 128-dimensional numerical vectors (embeddings) and compares them against the database to identify people.

Class: FaceRecognizer

- load_known_faces(): Loads all known face encodings from SQLite database
- add_new_face(name, image_path): Encodes a face from image and stores in DB
- add_face_from_frame(name, frame, location): Adds face from live camera
- recognize_faces(frame): Main function - detects and recognizes all faces
- save_recognition_snapshot(): Saves snapshot and logs to database
- draw_results(frame, results): Draws red boxes for matches, green for unknown

How Recognition Works

```
1. Resize frame to 25% (faster processing)
2. Convert BGR to RGB (face_recognition expects RGB)
3. Detect face locations using HOG detector
4. Generate 128-dim encoding for each face
5. Calculate Euclidean distance to all known faces
6. Find smallest distance (best match)
7. If distance < 0.6 = MATCH, else = UNKNOWN
8. Scale coordinates back to original size (x4)
```

7. Module 3: database.py

Purpose

Manages the SQLite database that stores known face encodings and detection logs. SQLite is used because it requires no separate server - the database is a single file.

Class: FaceDatabase

- create_tables(): Creates known_faces and detections tables
- add_known_face(name, encoding): Stores a face encoding as binary blob
- get_all_known_faces(): Returns all stored face encodings
- log_detection(name, confidence, camera_id, path, is_match): Logs each detection
- get_detection_history(limit): Returns recent detections
- get_known_face_count(): Returns total known faces
- delete_known_face(face_id): Removes a face from database

Database Schema

```
TABLE known_faces:
  id          INTEGER PRIMARY KEY
  name        TEXT NOT NULL
  encoding    BLOB (128-dim float64 vector)
  image_path  TEXT
  added_date  TEXT

TABLE detections:
  id              INTEGER PRIMARY KEY
  name            TEXT
  confidence      REAL
  camera_id       TEXT
  snapshot_path   TEXT
  timestamp       TEXT
  is_match        INTEGER (0 or 1)
```

8. Module 4: email_alert.py

Purpose

Sends automated email alerts with snapshot attachments when a known person (criminal/suspect) is detected by the system. Uses Python's smtplib with SMTP.

Class: EmailAlert

- send_alert(name, confidence, camera_id, snapshot_path, timestamp)
- test_connection(): Tests SMTP login before deployment

How Email Alert Works

- Step 1: System detects a known face (match found)
- Step 2: Snapshot is saved to snapshots/ folder
- Step 3: Email is composed with alert details
- Step 4: Snapshot is attached to the email
- Step 5: Email is sent via SMTP (Gmail)
- Step 6: Alert is logged to database

Email Configuration

To enable email alerts, set environment variables before running:

```
export SENDER_EMAIL="your_email@gmail.com"
export SENDER_PASSWORD="your_app_password"
```

Note: For Gmail, you need to generate an "App Password" from Google Account settings, not your regular password. This is required for SMTP access.

9. Module 5: detection_logger.py

Purpose

Logs every detection (match or unknown) to a CSV file for later review and analysis. Provides statistics and export functionality.

Class: DetectionLogger

- log(name, confidence, camera_id, snapshot_path, is_match): Write to CSV
- get_logs(limit): Read recent log entries
- get_stats(): Get total, matches, and unknown counts
- export_report(filepath): Export logs to a new CSV file

CSV Log Format

```
Timestamp,Name,Confidence,Camera_ID,Is_Match,Snapshot_Path
2026-08-19 14:30:22,John,0.92,CAM_01,1,snapshots/MATCH_John_20260819.jpg
2026-08-19 14:30:25,Unknown,0.00,CAM_01,0,
2026-08-19 14:30:28,Mary,0.87,CAM_01,1,snapshots/MATCH_Mary_20260819.jpg
```

10. Module 6: surveillance.py (Main System)

Purpose

The main integration script that wires all modules together into a single, working surveillance system. This is what you run to use the full system.

Class: SurveillanceSystem

- run(): Main loop - captures video, recognizes faces, sends alerts
- add_face_interactive(): Add a new face to the database during runtime
- show_logs(): Display detection log statistics

System Flow

```
1. Initialize FaceRecognizer, EmailAlert, DetectionLogger
2. Open webcam (camera index 0)
3. Display system info (known faces, email status)
4. LOOP:
   a. Read frame from webcam
   b. Detect and recognize faces
   c. Draw bounding boxes (red=match, green=unknown)
   d. If match found AND new detection:
      - Save snapshot
      - Send email alert
      - Log to database
   e. Display frame with info overlay
   f. Handle keyboard input (q/s/a/l/r)
5. On quit: Display session summary
```

Keyboard Controls

Key	Action
q	Quit the surveillance system
s	Save snapshot of current detections
a	Add a new face to the database (interactive)
l	Show detection log statistics
r	Refresh known faces from database

11. Module 7: register_faces.py

Purpose

Interactive tool to register known faces into the database. Supports registration from individual image files or entire dataset folders.

Options

- 1. Register from image file - Provide name and image path
- 2. Register from dataset folder - Auto-register all images in a person's folder
- 3. View registered faces - List all faces in the database
- 4. Delete a face - Remove a face by ID

12. How to Run the Complete System

Step 1: Register Known Faces First

Before running the surveillance system, register the people you want to track:

```
cd ~/Desktop/Desktop/major_project

# Option A: Register from an image file
python3 -m src.register_faces
# Select option 1, enter name, enter image path

# Option B: Register from dataset folder
# First collect dataset:
python3 -m src.dataset_collector john 50
# Then register:
python3 -m src.register_faces
# Select option 2, choose john
```

Step 2: Run the Surveillance System

```
python3 -m src.surveillance

# The system will:
# - Open webcam
# - Show green boxes around unknown faces
# - Show red boxes around known/matched faces
# - Send email if a match is found
# - Log all detections
```

Step 3: Enable Email Alerts (Optional)

```
export SENDER_EMAIL="your_email@gmail.com"
export SENDER_PASSWORD="your_app_password"
python3 -m src.surveillance
```

Step 4: Test Recognition on an Image

```
python3 -m src.test_recognition path/to/photo.jpg
```

13. How Face Recognition Works (CNN + Embeddings)

What is a CNN?

A Convolutional Neural Network (CNN) is a type of deep learning model specifically designed for processing images. It learns to recognize patterns like edges, textures, and shapes by passing filters (kernels) over the image.

What are Face Embeddings?

A face embedding is a 128-dimensional numerical vector that represents a face. Think of it as a "face fingerprint" - a unique set of 128 numbers that describes the face. Similar faces produce similar vectors.

How Comparison Works

```
Person A embedding: [0.12, -0.34, 0.56, ...] (128 numbers)
Person B embedding: [0.11, -0.33, 0.55, ...] (128 numbers)

Euclidean Distance = sqrt(sum((A[i] - B[i])^2))

Distance < 0.6  --> SAME PERSON (match)
Distance >= 0.6 --> DIFFERENT PERSON (no match)

Lower distance = more similar faces
0.0 = identical, 0.6+ = completely different
```

The dlib ResNet Model

The model used is a ResNet (Residual Network) trained on 3 million face images. It learns to map any face to a point in 128-dimensional space where faces of the same person are close together and faces of different people are far apart. This is called a "siamese network" training approach.

14. Testing Results

Module Tests

All 8 modules were tested successfully:

- [OK] Database module - SQLite create, insert, read, delete
- [OK] FaceRecognizer - Load known faces, encode, compare
- [OK] EmailAlert - Initialize, check configuration
- [OK] DetectionLogger - CSV write, read, statistics
- [OK] SurveillanceSystem - Import and initialize
- [OK] register_faces - Import and initialize
- [OK] FaceDetector (Week 1) - Haar Cascade loaded
- [OK] DB insert + read - Test encoding stored and retrieved

Test Commands

```
# Test all modules
cd ~/Desktop/Desktop/major_project
python3 -c "from src.database import FaceDatabase; print("DB OK")"
python3 -c "from src.face_recognizer import FaceRecognizer; print("FR OK")"
python3 -c "from src.email_alert import EmailAlert; print("EA OK")"
python3 -c "from src.detection_logger import DetectionLogger; print("DL OK")"
python3 -c "from src.surveillance import SurveillanceSystem; print("SS OK")"
```

15. Progress Summary & Remaining Work

Completed (60%)

- Face detection using OpenCV Haar Cascade
- Face recognition using dlib CNN embeddings
- SQLite database for face encodings
- Automated email alert system
- CSV detection logging
- Interactive face registration
- Main surveillance system integration
- Week 1 documentation

Remaining (40%)

- Flask web dashboard for live monitoring
- Multi-camera support and tracking
- Behavior analysis (loitering, zone intrusion)
- YOLO object detection for weapons/objects
- Anti-spoofing (block photo/video attacks)
- Testing with real-world scenarios
- Performance optimization
- Final documentation and report

Next Steps

- Week 3: Build Flask dashboard with live video feed
- Week 4: Add multi-camera and YOLO detection
- Week 5: Add behavior analysis and anti-spoofing
- Week 6: Testing, optimization, final report